

14.02.26
Левина.

- символьная информация — debug information.

Системные методы оптимизации приложений.

→ оптимизация кода на уровне высокого уровня при его компиляции.

- ! не меняют исходного кода но накладывают определённые условия.

• High Level Optimizer (HLO)

- ориентация на макс. скорость работы (может переписать алгоритм, — ОЗ)

• Разворачивание циклов (loop unrolling)

• Ограничения:

- цикл должен удовл. требованиям (или с вектор-из)
- HLO не обязательно повышает производительность.
- увеличивает размер исполн-го кода

• Interprocedural Optimization (IPO)

→ Функции оптимизируются внутри файлов.

- Inlining

"Метипроцедурная оптимизация"

- много файловое ветвление

- замена кода

- разделение данных

Source Files → Compile with IPO → .o files with IPO → Link with IPO → exe.

• Ограничения:

- IPO может увеличить размер кода (что может повлиять на использование кэша процессора)

- компилятор может быть сильно медленнее

• Опции: -ip (между процедурами)

-ipo (-ip + между сек-ми файлами)

gcc: -fipa

• Векторная оптимизация

Profile Guided Optimization (PGO)

(динамич-ая, замкнутая, с обратной связью)

Суть:

1. Инструментальная компиляция:
(Instrumented)

сбор статистики,

2. Инструмент-е выполнение
(на целевой платформе)

типичная нагрузка

3. Компиляция и компоновка
с обратной связью.

Матри:

① Compile
with PGO

② Run Instb. app

создаются
динамич-е
файлы
(профили)

③ Оптим-за
подин-на
с. обратн
связью.

Оптимизатор: gcc: -fprofile-generate
-fprofile-use)

Intel: -prof-gen
-prof-use

- Охраняет типичной ред. нагрузки.

— эти методы не работают

→ следует понимать, что и когда использовать.

- gcc - match = native (сб-на не у платформ)

Основы векторизации кода

Михаил,
21.02.26.

- один из показателей качества кода

- производительность $P_{\text{perf}}(\text{code performance})$

- произв-ть уместна кода

$$P_{\text{perf}} = 1 / T [c^{-1}]$$

- потребл.-мая память Mem
(memory consumption)

- особенности современных микроархитектур.

- сложность кода $Comp(\text{code complexity})$

$$Comp = LOC (\text{lines of code})$$

$$= Cycl (\text{суммарная сложность})$$

- кол-во инструкций по раз-м маршрутов.

относится к разработке.

— Эффективность работы

$P_{\text{prod}} (\text{code productivity})$

$$P_{\text{prod}} = \frac{P_{\text{perf}}}{\text{Compl}} = \frac{1}{T \cdot \text{Compl}}$$

— интерпретируемый показатель.

• Высоко-эффективные технологии

(high-productivity computing)

— Chapel (Cray)

— X10 (IBM)

— Fortran 2023 (ISO/IEC)

→ Совмещение микроархитектур

тип: SIMD (Single Instruction on multiple data)

• Эволюция архитектур

— Эффективность процессора

— P_{perf}

— Рассчитываемая мощность TDP

(Thermal design power)

$$P_{\text{prod}} = P_{\text{perf}} / \text{TDP}$$

— Hyper-threading — гиперпоточность

Эффект. Загрузка ядра

Архитектура SIMD современных ядер

- представлять файл

↓
АЛУ
↓
ОЗУ

(Scalar Unit)

Vector Unit - глоб. регистр. Register File

— Intel SIMD ISA Evolution

IA-32 - 1983 год, 32 рег. ячеек.

IA-64, Windows XP-64

AMD-64 - расширение

x86 (x - цифра, номер ячеек)

MMX, 64 бита, 1 App, Pentium 5

3DNow! (AMD), 64 бита, VIA

SSE2, модификация (в том числе)

(Streaming SIMD Extension)

SSE4 - 128 бит

AVX - 256 бит - 8 ячеек с мал. записью

AVX - 512.

• Со временем размер векторных регистров только увеличивается. в 2 раза

• Примеры векторизации

- 4 op-и по 3 операция → одна с 8-ю операциями
- измат-е порядка вычисления (необяз-но произ- и нужной точности)
- векторизация $\neq$ оптимизация.
- стоит всегда извлекаться от зависимости итерий после записи!
- из-за условий внутри цикла может не векторизоваться.

Регулярный доступ и память -

Минус
7.03.26

- это доступ и элементы в порядке
их расположения в памяти.

- Он необходим для спlicing и
иэм-промахов

- иэм-промахов - не имеют значения в
иэм-памяти, 1, 2, 3 уровня.

- управляемая векторизация: Оран MP.

- Как обеспечить гарантию отсутствия
перекрываний (overlap) памяти?

- unique - ptr.

- Как эффективно проводить сложение
и умножение матриц на современных арх.
- выделение первого эл-та и строки
(сложение)

• эффекивная векторизация

- выравнивание
- отсутствие зависимостей м/у строками
- регулярный доступ.

о нет технологий с хранения
по строкам / столбцам!

- в случае изменения
порядка строк / столбцов
стоит оставлять комментарии в коде

Условия векторизации

1) Данные должны быть сложными
в памяти

2) Должен обеспечиваться регулярный
доступ к памяти

3) Должен быть баланс:

- первый элемент строки
- последний элемент строки

4) не должно быть перерывов по высоте (исевдотомов)

5) не должно быть завышенных потерь в виде тепла - вода - газы

• Спорение

Легенда 2
7.03.26

- за счёт векторизации
- многоядерности и многопоточности.

• SVE 1, 2; SME - рассмотрены

1. Использовать следующие данные

- статические массивы
- динамические массивы
- динамически изменяющиеся массивы

Salomon (C++)

→ динамическая структура - в памяти/сигнале.

2. Обеспечивать регулярный доступ
и память.

- проходить э-тс в порядке их
расположения

- переносить доступ -

через оператор ввода / вывода

- обработка данных: рел-и

3. Обеспечивать исправление
ошибок

4. Обеспечивать гарантию отсутствия
перерывов по памяти

5. Обеспечивать гарантию

отсутствия зависимости между процессами
(включая параллелизацию)

6. Использовать высший уровень абстракции и регулярное программирование

7. Обеспечивать переносимость исполняемого кода и/у целевыми платформами